

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1710

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

ASSESSMENT TOOL 3 — REVERSE ENGINEERING

Student	Register Number	Assigned System
M ARSHINI	192424322	Warehouse Robotics Coordination System

REVERSE ENGINEERING A WAREHOUSE ROBOTICS COORDINATION SYSTEM

M ARSHINI | Register Number: 192424322

1. Problem Understanding and Observed Behaviour

The target system is a large fulfillment warehouse in which hundreds of autonomous mobile robots (AMRs) move inventory between storage locations, picking stations, packing areas and charging points. The observed behaviour suggests that robots continuously receive tasks, select routes, avoid one another and report their status to a central coordination layer. The system must make decisions quickly while maintaining throughput, avoiding collisions and continuing operation when individual devices or processing nodes fail.

2. Reverse-Engineered Architecture

A practical reconstruction is a layered distributed architecture. Robot sensors and warehouse systems produce events; a streaming ingestion layer collects them; PySpark processes historical and near-real-time records; an RDD-based analytics layer derives operational features; and an intelligent-agent layer converts those features into task-allocation and path-planning decisions.

Layer	Main Components	Purpose
Data Sources	Robot telemetry, LiDAR/ultrasonic sensors, barcode/RFID scans, task queue, shelf status	Capture robot position, obstacles, inventory and work demand
Ingestion	Message queue / event gateway	Collect events from hundreds of robots and warehouse services
Distributed Processing	PySpark cluster + RDDs	Clean, partition, transform and aggregate large sensor/task logs
AI / Analytics	Congestion prediction, workload scoring, route-cost estimation	Estimate demand, risks and efficient assignments
Intelligent Agents	Robot agents, task manager, traffic coordinator, exception agent	Make local and global decisions and coordinate robots
Execution	Robot control API + warehouse management system	Dispatch tasks, update routes and record outcomes

3. Likely Data Flow

Robot sensors + task queues → event ingestion → RDD creation → filtering/cleaning → feature extraction → distributed analytics → agent coordination → task/route decision → robot execution → telemetry feedback → continuous update.

PYSPARK AND RDD-BASED PROCESSING

4. How Large-Scale Warehouse Data Is Processed

The warehouse generates a large number of records every second. Instead of processing these records on one machine, PySpark distributes partitions of the data across cluster workers. RDDs are useful for immutable, fault-tolerant processing of telemetry and historical task logs. If a worker fails, lost partitions can be recomputed from the RDD lineage rather than requiring the entire computation to restart.

RDD Operation	Warehouse Use	Example
textFile / parallelize	Create distributed records from logs or batches	Load robot telemetry records
filter()	Remove invalid or irrelevant events	Keep records with valid robot IDs and timestamps
map()	Convert raw events into features	Convert (robot, x, y, speed) into a structured movement record
flatMap()	Expand nested sensor/task information	Extract multiple obstacle events from one scan
mapToPair()	Create key-value records	(zone, congestion_event)
reduceByKey()	Aggregate distributed measurements	Count pending tasks per warehouse zone
groupByKey()	Collect records by robot/zone when required	Group route events by robot ID
sortByKey()	Order tasks by a priority-related key	Sort jobs by deadline or urgency
join()	Combine datasets with common identifiers	Join robot status with assigned task data
cache()	Reuse frequently accessed data	Keep active robot-state RDD in memory
count()/collect()	Produce operational summaries	Count delayed tasks or inspect a small diagnostic result

5. Example Distributed Processing Logic

Suppose every robot produces records of the form (robot_id, zone, battery, speed, obstacle_count, task_id). The system can filter faulty readings, map each record into a risk score, partition records by zone, and reduce them to calculate congestion and workload. The resulting zone-level RDD can be joined with the task queue so that agents receive both demand information and robot availability.

Illustrative flow: telemetryRDD → filter(valid) → map(features) → keyBy(zone) → reduceByKey(aggregate) → join(taskRDD) → agent decision layer.

INTELLIGENT AGENT COORDINATION

6. Agent Architecture

The reconstructed system can be viewed as a multi-agent system. Each robot acts as a local autonomous agent, while supervisory agents coordinate tasks at warehouse level. Agents exchange compact state information rather than sending all raw sensor data to every robot.

Agent	Responsibilities	Inputs	Outputs
Robot Agent	Local navigation, obstacle avoidance, battery awareness	Position, nearby obstacles, assigned job	Velocity/route action, status update
Task Allocation Agent	Select suitable robot for each pending task	Task priority, robot availability, distance, battery	Robot-task assignment
Traffic Coordination Agent	Prevent congestion and route conflicts	Zone occupancy, predicted traffic, planned routes	Route constraints / rerouting commands
Inventory Agent	Track movement of items and storage locations	Barcode/Rfid events, WMS records	Inventory state updates
Charging Agent	Schedule charging without reducing throughput	Battery levels, charger occupancy, task urgency	Charging schedule
Exception Agent	Handle failures and unusual situations	Fault logs, missed deadlines, blocked paths	Escalation, task reassignment

7. Agent Interaction Scenario

Assume a picking zone suddenly receives many orders. The Task Allocation Agent detects a growing queue from the distributed task RDD. It requests current robot availability from Robot Agents and receives position, battery and estimated completion time. The Traffic Coordination Agent checks whether assigning several robots to the same aisle would create congestion. It then recommends a balanced allocation. If one robot reports a blocked path, the Exception Agent removes that robot from the active assignment and sends the task back to the allocation queue. This produces decentralized reactions while retaining global coordination.

8. Decision Logic

Decision Factor	Higher Priority When	Effect on Decision
Task urgency	Deadline is close	Prefer robots with short estimated completion time
Distance	Robot is near pickup point	Reduce travel time
Battery	Battery is healthy	Allow longer tasks; avoid unnecessary charging
Congestion	Zone is crowded	Choose alternate route or robot
Robot health	Fault indicators appear	Remove robot and reassign work
Workload balance	One zone has a large queue	Send available robots while respecting traffic limits

FAULT TOLERANCE, SCALABILITY AND DATA PIPELINE

9. Why RDD Fault Tolerance Matters

Warehouse coordination cannot depend on a single processing node. RDD lineage provides a recovery mechanism for lost partitions. If a worker holding part of a telemetry or task RDD fails, Spark can recompute the affected partition from its parent transformations. Checkpointing or persistent storage can additionally be used for long-running pipelines.

- Partitioning distributes telemetry and task records across multiple workers.
- Lineage allows failed partitions to be reconstructed without recreating the complete dataset.
- Caching improves response time for repeatedly accessed robot-state data.
- Broadcast variables can distribute small read-only configuration such as warehouse maps or safety limits.
- Key-based partitioning keeps related zone or robot records together for aggregation.
- Supervisory agents can continue operating when an individual robot becomes unavailable.

10. End-to-End Processing Pipeline

Stage	Processing	Distributed-AI Contribution
1. Collection	Robots, scanners and WMS emit events	Many devices generate events concurrently
2. Ingestion	Events enter the distributed processing environment	Prevents a single collector from becoming a bottleneck
3. Cleaning	Remove corrupt records and normalize timestamps/IDs	Parallel filtering across partitions
4. Feature Extraction	Compute speed, distance, battery risk, queue length and congestion	map/flatMap transformations
5. Aggregation	Calculate zone workload, robot availability and delay statistics	reduceByKey / aggregations
6. Decision Preparation	Join task and robot-state information	Distributed joins combine large datasets
7. Agent Decision	Agents allocate tasks and select routes	Combines global analytics with local autonomy
8. Execution	Commands are sent to robots	Low-latency control interface
9. Feedback	New telemetry records are generated	Closed loop for continuous adaptation

11. Scalability Strategy

- Horizontal scaling: add Spark worker nodes as robot and event volume increases.
- Partition by robot ID or warehouse zone to distribute computation evenly.
- Separate high-frequency telemetry processing from slower historical analytics.
- Use caching for active-state datasets and persistent storage for historical logs.
- Keep safety-critical collision avoidance local to robots so it does not depend entirely on cluster latency.
- Use hierarchical coordination: local robot agents manage immediate motion while supervisory agents manage warehouse-wide optimization.

CRITICAL ANALYSIS AND INDUSTRIAL APPLICABILITY

12. Strengths of the Reconstructed Design

- Distributed processing prevents a central server from becoming the only computational bottleneck.
- RDD transformations are well suited to large historical telemetry and task-log analysis.
- Multi-agent coordination allows local decisions to be made close to the robots while preserving system-wide optimization.
- Fault tolerance is improved because Spark can recompute lost partitions and agents can reassign interrupted tasks.
- The architecture separates analytics, coordination and physical robot control, making the system easier to maintain.

13. Limitations and Risks

- RDD-based batch processing alone may not satisfy strict millisecond-level control requirements; safety-critical motion should remain at the edge.
- Network delay or temporary communication loss can make a global route decision stale.
- Poor sensor calibration can lead to incorrect congestion or obstacle estimates.
- A biased task-priority policy may repeatedly favour some warehouse zones or robots.
- As the number of robots increases, coordination traffic and route conflicts can grow rapidly.
- Cybersecurity is essential because compromised robot commands could affect physical equipment.

14. Industry Deployment Challenges

Challenge	Impact	Mitigation
Data volume	Telemetry grows with every robot and sensor	Partitioning, aggregation, retention policies
Latency	Global analytics may be slower than local control	Keep immediate safety decisions at edge
Heterogeneous robots	Different models expose different interfaces	Common robot API and normalized event schema
Failures	Battery, sensor or motor faults interrupt tasks	Health monitoring and automatic reassignment
Security	Unauthorized commands can cause physical damage	Authentication, encryption, access control and audit logs
Safety	Robot collisions can injure workers or damage inventory	Local obstacle avoidance, geofencing and fail-safe states
Scalability	More robots increase coordination complexity	Hierarchical agents and zone-based scheduling

15. Performance Measures to Validate the Reverse-Engineered System

- Average task completion time and percentage of tasks completed before deadline.
- Robot utilization and idle time.
- Average travel distance per completed task.
- Queue waiting time by warehouse zone.
- Collision/near-collision rate and blocked-path frequency.
- Cluster processing latency and throughput for telemetry records.

INNOVATION, CO5 MAPPING AND CONCLUSION

16. Proposed Enhancement

A useful enhancement is a hierarchical predictive multi-agent scheduler. The PySpark layer would learn historical patterns such as hourly order surges, frequently congested aisles and charging demand. A forecasting component would produce zone-level workload predictions, while a supervisory agent would reserve capacity in advance. Robot agents would still retain local collision avoidance and could override a planned route whenever a real-time obstacle is detected.

This enhancement is feasible because it reuses the existing distributed telemetry and task logs. The expected impact is lower queueing time, fewer emergency reroutes and better battery utilization during predictable demand peaks. The design also reduces the need for every robot to participate in a single global optimization problem.

17. CO5 Concept Mapping

CO5 Concept	Application in This Design
PySpark	Distributed processing engine for warehouse telemetry, task and historical logs.
RDD Operations	filter, map, flatMap, reduceByKey, join, sortByKey and cache support distributed transformations and aggregation.
Intelligent Agents	Robot, task-allocation, traffic, charging, inventory and exception agents make coordinated decisions.
Scalability	Cluster workers and partitioned RDDs allow processing to grow with robot/event volume.
Distributed AI Integration	Analytics, agents, warehouse software and physical robots operate as integrated components.
Fault Tolerance	RDD lineage and task reassignment allow recovery from worker or robot failures.

18. Conclusion

The warehouse robotics system can be reverse engineered as a distributed AI platform combining continuous robot telemetry, PySpark-based large-scale processing, RDD transformations and a multi-agent coordination layer. RDDs support parallel analysis of sensor and task-queue logs and provide lineage-based recovery when processing nodes fail. Intelligent agents translate the processed information into task allocation, routing, charging and exception-handling decisions. The most important architectural principle is to combine centralized distributed analytics with decentralized local robot control: the cluster optimizes warehouse-wide operations, while each robot retains fast safety decisions. This structure provides a practical path toward scalable, fault-tolerant and intelligent warehouse automation.

19. Overall Assessment

Problem Understanding: The operational objective, constraints and observable behaviour are clearly identified.

Engineering Concepts: PySpark, RDD transformations, distributed processing, fault tolerance and intelligent-agent concepts are integrated.

Solution Design: The reconstructed architecture is practical, scalable and separates safety-critical local control from global optimization.

Analysis: Performance indicators, risks and deployment challenges are explicitly identified for validation.